

CAMPUS SEGURO

C5 de Emergencia - ESP32

1. Descripción General

El sistema es un botón de emergencia basado en ESP32 diseñado para instalarse en pasillos o edificios universitarios. Permite al usuario reportar incidentes de seguridad pulsando un botón físico un número de veces determinado; cada conteo de pulsaciones activa un tipo de emergencia diferente.

La alerta se publica via MQTT con toda la información del dispositivo (ID, zona, coordenadas GPS fijas y tipo de emergencia) hacia un servidor de vigilancia C5. El botón retroalimenta al usuario mediante un LED RGB, un buzzer y una pantalla LCD I2C de 16x2.

2. Hardware y Conexiones

2.1 Diagrama Esquemático (Proteus).

El siguiente esquema muestra la conexión de todos los componentes: ESP32 DevKit, LED RGB con ánodo común, transistor 2N2222A para el buzzer, LCD 16x2 via módulo I2C PCF8574 y pulsador.

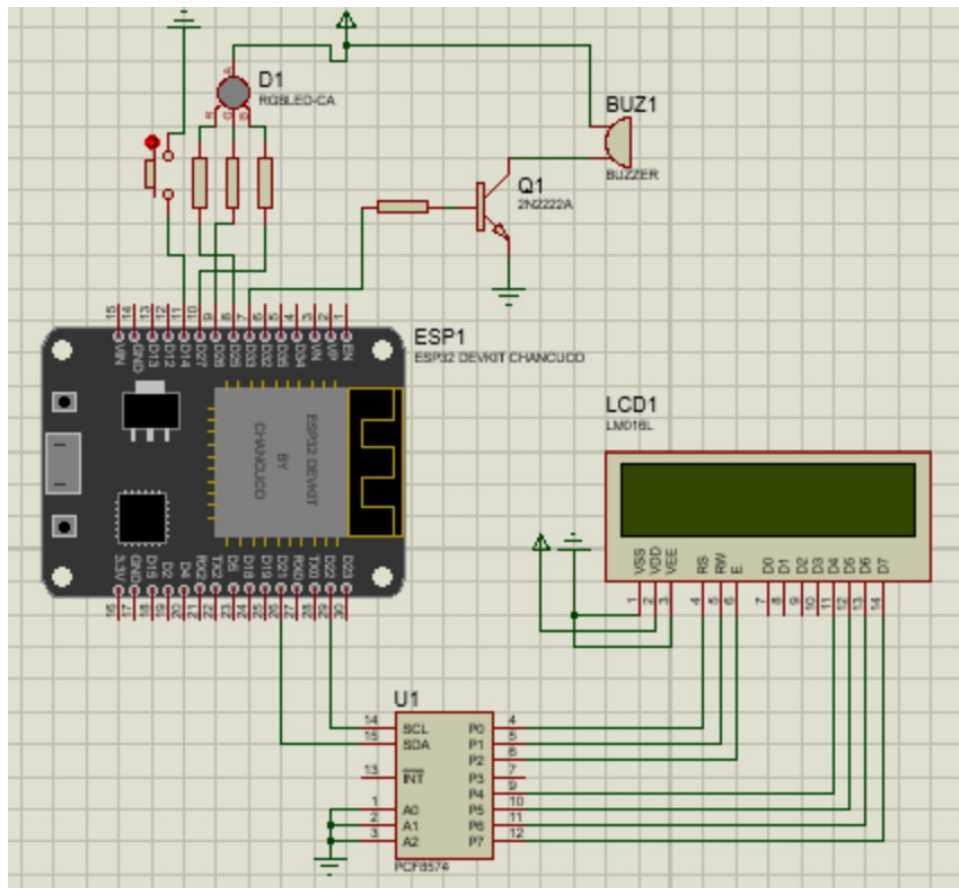


Figura 1 - Esquema electrónico en Proteus

2.2 Diagrama de Conexión Real (Wokwi)

Las figuras 2, 3 y 4 muestran el circuito real simulado en Wokwi con los distintos estados del sistema (reposo, conectando al servidor y sistema listo).

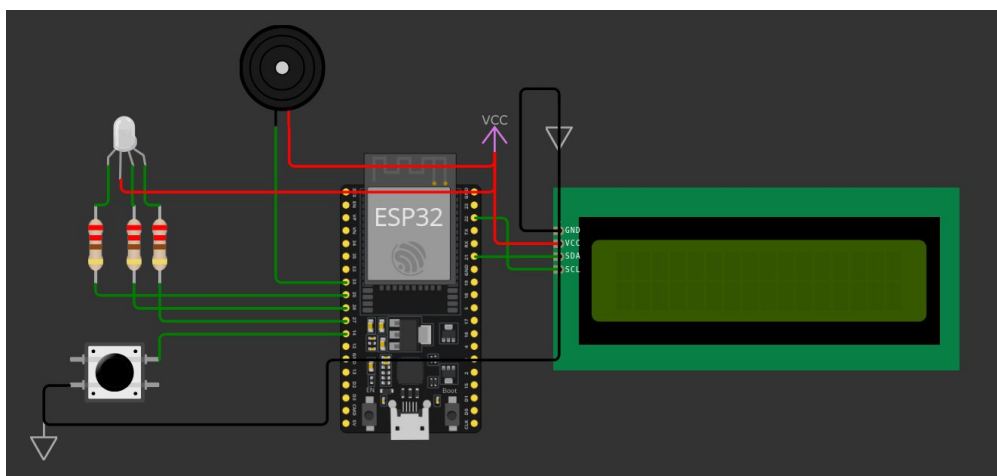


Figura 2 - Circuito en reposo (sin conexión)

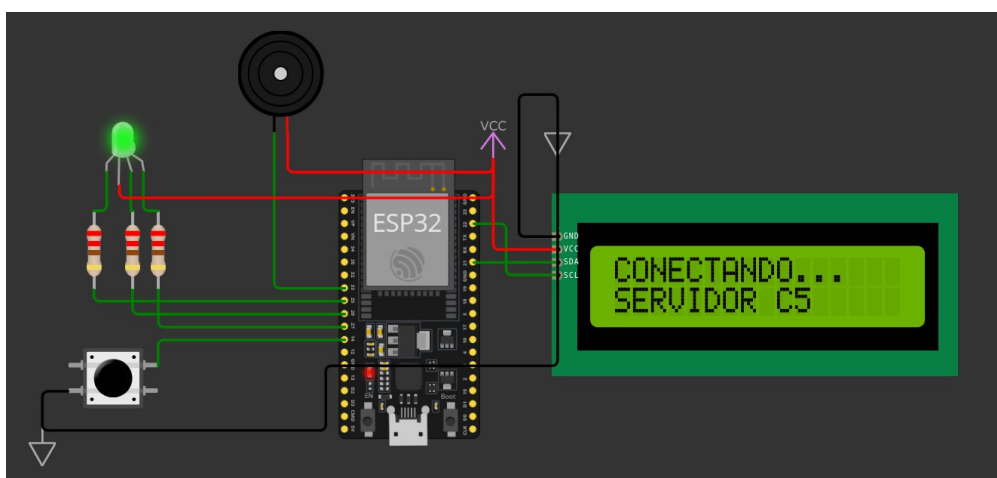


Figura 3 - Estado: Conectando al servidor C5

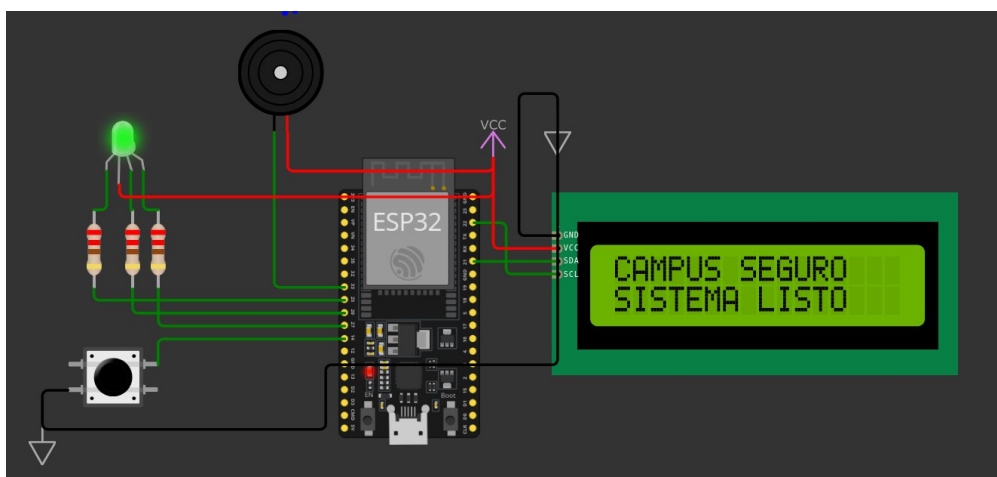


Figura 4 - Estado: Sistema listo (LED verde)

Nota sobre la Simulación (Figuras 2, 3 y 4): Debido a que el entorno de simulación Wokwi no evalúa los límites de corriente física, el zumbador (Buzzer) se ilustra conectado directamente

al pin GPIO 33 para comprobar la lógica de programación del sistema. Sin embargo, para el ensamblaje físico del hardware, es **estrictamente necesario** implementar la etapa de potencia con el transistor NPN 2N2222A (como se detalla en el diagrama esquemático de Proteus de la Figura 1). Un pin digital del ESP32 no suministra la corriente requerida por el zumbador, por lo que una conexión física directa podría dañar el microcontrolador de forma permanente.

2.3 Pines Utilizados

Pin GPIO	Componente
GPIO 14	Boton pulsador
GPIO 25	LED RGB - Rojo
GPIO 26	LED RGB - Verde
GPIO 27	LED RGB - Azul
GPIO 33	Buzzer (via 2N2222A)
GPIO 21 (SDA)	LCD I2C PCF8574
GPIO 22 (SCL)	LCD I2C PCF8574

3. Dependencias y Bibliotecas

El firmware utiliza las siguientes bibliotecas de Arduino. Todas estan disponibles desde el gestor de bibliotecas de Arduino IDE o PlatformIO.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

4. Configuración del Sistema.

4.1 Credenciales de Red y MQTT

Las constantes de red se definen al inicio del archivo. Modificar estos valores para adaptar el boton del c5 a otra instalacion.

```
const char* WIFI_SSID      = "SpaceX";
const char* WIFI_PASSWORD  = "Isic2026??$";
const char* MQTT_BROKER    = "192.168.2.46";
const int   MQTT_PORT      = 1883;
const char* MQTT_TOPIC     = "c5/alerts/panic";
```

4.2 Identificación del Dispositivo

Identidad del totem

```
const char* DEVICE_ID      = "PASILLO";  
const char* LOCATION_NAME = "EDIFICIOS";
```

Coordenadas GPS fijas (el totem no tiene GPS móvil)

```
const float FIXED_LAT = 19.912752379357503;  
const float FIXED_LON = -99.57860148464074;
```

4.3 Tipos de Emergencia

El sistema soporta 5 tipos de emergencia. El número de pulsaciones del botón determina cuál se activa (índice = (pulsaciones - 1) % 5).

```
const char* EMERGENCY_TYPES[] = {  
    "robo_con_violencia",  
    "acoso_violencia",  
    "incendio",  
    "emergencia_medica",  
    "sospechoso"  
};
```

```
const int NUM_TYPES = 5;
```

Texto corto para el LCD (max 16 caracteres)

```
const char* EMERGENCY_LCD[] = {  
    "ROBO / ASALTO",    // Crítico  
    "ACOSO/VIOLENCIA", // Alto  
    "INCENDIO / GAS",  // Crítico  
    "EMERG. MEDICA",   // Alto  
    "SOSPECHOSO"       // Medio  
};
```

5. Funciones Auxiliares

5.1 actualizarPantalla()

Limpia el LCD y escribe dos lineas de texto. Maximo 16 caracteres por linea.

```
void actualizarPantalla(String linea1, String linea2) {  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print(linea1);  
    lcd.setCursor(0, 1);  
    lcd.print(linea2);  
}
```

5.2 setColor()

Controla el LED RGB de anodo comun. Logica INVERTIDA: pasar true enciende el canal (escribe LOW al pin).

LED RGB con anodo comun: LOW = encendido, HIGH = apagado

```
void setColor(bool r, bool g, bool b) {  
    digitalWrite(RED_PIN, !r);  
    digitalWrite(GREEN_PIN, !g);  
    digitalWrite(BLUE_PIN, !b);  
}
```

Uso:

setColor -> Rojo (alerta enviada)

setColor -> Verde (sistema listo)

setColor -> Amarillo (conectando WiFi)

setColor -> Magenta (revisando, parpadea)

setColor-> Apagado

5.3 Funciones de Buzzer

El buzzer emite patrones sonoros distintos para cada evento. El transistor 2N2222A amplifica la corriente necesaria.

```
void beep(int duracion_ms) {  
    digitalWrite(BUZZER_PIN, HIGH);  
    delay(duracion_ms);  
    digitalWrite(BUZZER_PIN, LOW);  
    delay(50);  
}
```

6. Funciones de Comunicacion

6.1 connectWiFi()

Conecta el ESP32 a la red WiFi. Bloquea la ejecucion hasta obtener conexion.

```
void connectWiFi() {
    Serial.print("[WiFi] Conectando a red campus");
    actualizarPantalla("CONECTANDO...", "RED CAMPUS");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500); Serial.print(".");
    }
    Serial.printf("\n[WiFi] ✓ IP: %s\n", WiFi.localIP().toString().c_str());
}
```

6.2 connectMQTT()

Conecta al broker MQTT, publica un mensaje de presencia en c5/devices/online y se suscribe al topic de estado para recibir confirmaciones del C5.

```
void connectMQTT() {
    int attempts = 0;
    actualizarPantalla("CONECTANDO...", "SERVIDOR C5");

    while (!mqttClient.connected() && attempts < 5) {
        Serial.print("[MQTT] Conectando...");
        if (mqttClient.connect(DEVICE_ID)) {
            Serial.println(" ✓");

            StaticJsonDocument<128> onlineDoc;
            onlineDoc["device_id"] = DEVICE_ID;
            onlineDoc["location_name"] = LOCATION_NAME;
            onlineDoc["status"] = "online";
            char onlineBuf[128];
            serializeJson(onlineDoc, onlineBuf);
            mqttClient.publish("c5/devices/online", onlineBuf);

            mqttClient.subscribe(TOPIC_STATUS);
            Serial.printf("[MQTT] Suscrito a: %s\n", TOPIC_STATUS);
        } else {
            Serial.printf(" x rc=%d\n", mqttClient.state());
            delay(2000); attempts++;
        }
    }
}
```

6.3 mqttCallback()

Callback invocado automaticamente por PubSubClient al recibir un mensaje. Procesa el campo status del JSON y actualiza el estado del totem.

```
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    String mensaje = "";
    for (unsigned int i = 0; i < length; i++) mensaje += (char)payload[i];

    Serial.printf("[MQTT] Mensaje recibido en %s: %s\n", topic, mensaje.c_str());

    StaticJsonDocument<200> doc;
    if (deserializeJson(doc, mensaje) != DeserializationError::Ok) return;

    const char* status = doc["status"];
    if (!status) return;

    if (strcmp(status, "reviewing") == 0) {
        Serial.println("[Status] Guardia revisando incidente");
        alertaEnRevision = true;
        actualizarPantalla("GUARDIA:", "REVISANDO...");
    } else if (strcmp(status, "confirmed") == 0) {
        Serial.println("[Status] EMERGENCIA CONFIRMADA – guardia en camino");
        alertaEnRevision = false;
        setColor(0, 1, 0);
        beepAtendida();
        actualizarPantalla("GUARDIA EN CAMINO", "MANTENTE SEGURO");
        tiempoResolucion = millis();
    } else if (strcmp(status, "false_alarm") == 0) {
        Serial.println("[Status] Marcado como falsa alarma");
        alertaEnRevision = false;
        setColor(0, 1, 0);
        beep(200);
        actualizarPantalla("ESTADO:", "FALSA ALARMA");
        tiempoResolucion = millis();
    }
}
```

7. Envio de Alerta - sendAlert()

Funcion central del sistema. Construye el payload JSON y lo publica en c5/alerts/panic con retained=true para que el servidor no pierda la alerta si esta temporalmente desconectado.

```
void sendAlert(const char* emergencyType, const char* lcdLabel) {
    Serial.printf("\n[ALERT] Enviando: %s\n", emergencyType);
    alertaEnRevision = false;
    setColor(1, 0, 0);

    actualizarPantalla(lcdLabel, "ENVIANDO...");

    StaticJsonDocument<512> doc;
    doc["device_id"]      = DEVICE_ID;
    doc["location_name"]  = LOCATION_NAME;
    doc["latitude"]       = FIXED_LAT;
    doc["longitude"]      = FIXED_LON;
    doc["emergency_type"] = emergencyType;
    doc["location_method"] = "totem_fijo";
    doc["campus"]         = true;

    char buffer[512];
    serializeJson(doc, buffer);

    if (!mqttClient.connected()) connectMQTT();

    if (mqttClient.publish(MQTT_TOPIC, buffer, true)) {
        Serial.println("[ALERT] ✓ Alerta enviada al centro de vigilancia");
        beepAlertaEnviada();
        actualizarPantalla("ALERTA ENVIADA", "ESPERANDO...");
    } else {
        Serial.println("[ALERT] x Error de envío");
        beepError();
        actualizarPantalla("ERROR", "FALLO EL ENVIO");
        for (int i = 0; i < 6; i++) {
            setColor(1, 0, 0); delay(150);
            setColor(0, 0, 0); delay(150);
        }
    }

    setColor(0, 1, 0);
}
```

Construccion del payload JSON

Ejemplo de payload JSON generado:

```
{
  "device_id":    "PASILLO",
  "location_name": "EDIFICIOS",
  "latitude":     19.912752379357503,
  "longitude":    -
  99.57860148464074,
  "emergency_type": "incendio",
  "location_method": "totem_fijo",
  "campus":        true
}
```

8. Funcion setup()

Se ejecuta una sola vez al encender el ESP32. Inicializa perifericos, conecta a WiFi y MQTT, y deja el totem en estado listo.

```
void setup() {
  Serial.begin(115200);

  lcd.init();
  lcd.backlight();
  actualizarPantalla("CAMPUS SEGURO", "INICIANDO...");

  Serial.println("\n┌──────────────────────────────────────────┐");
  Serial.println("│ Campus Seguro – Tótem de Emergencia │");
  Serial.printf("│ ID: %-33s │\n", DEVICE_ID);
  Serial.printf("│ Zona: %-31s │\n", LOCATION_NAME);
  Serial.println("└──────────────────────────────────────────┘");

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  snprintf(TOPIC_STATUS, sizeof(TOPIC_STATUS), "c5/alerts/status/%s", DEVICE_ID);

  setColor(1, 1, 0);

  connectWiFi();
  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(mqttCallback);
  connectMQTT();

  setColor(0, 1, 0);
  beep(200);

  actualizarPantalla("CAMPUS SEGURO", "SISTEMA LISTO");

  Serial.println("\n[Setup] ✓ Tótem listo");
  Serial.printf("[Setup] Zona: %s\n", LOCATION_NAME);
  Serial.println("[Setup] Pulsaciones:");
  Serial.println("1 → robo_con_violencia (CRÍTICO – 2 min)");
  Serial.println("2 → acoso_violencia (ALTO – 5 min)");
  Serial.println("3 → incendio (CRÍTICO – 2 min)");
  Serial.println("4 → emergencia_medica (ALTO – 5 min)");
  Serial.println("5 → sospechoso (MEDIO – 15 min)\n");
}
```

9. Funcion loop()

El bucle principal corre cada 20 ms. Gestiona la reconexion automatica, mensajes MQTT entrantes, parpadeo del LED y la logica de deteccion de pulsaciones con ventana de tiempo.

```
void loop() {  
  if (WiFi.status() != WL_CONNECTED) connectWiFi();  
  if (!mqttClient.connected()) connectMQTT();  
  mqttClient.loop();  
  
  if (tiempoResolucion > 0 && (millis() - tiempoResolucion > 5000)) {  
    actualizarPantalla("CAMPUS SEGURO", "SISTEMA LISTO");  
    tiempoResolucion = 0;  
  }  
  
  if (alertaEnRevision) {  
    unsigned long now = millis();  
    if (now - lastBlinkTime > 500) {  
      lastBlinkTime = now;  
      blinkState = !blinkState;  
      blinkState ? setColor(1, 0, 1) : setColor(0, 0, 0);  
    }  
  }  
  
  bool buttonPressed = (digitalRead(BUTTON_PIN) == LOW);  
  if (buttonPressed && !buttonWasPressed) {  
    unsigned long now = millis();  
    if (now - lastButtonPress > DEBOUNCE_MS) {  
      lastButtonPress = now;  
      buttonWasPressed = true;  
      if (pressCount == 0) pressWindowStart = now;  
      pressCount++;  
      Serial.printf("[Button] Pulsación #%d\n", pressCount);  
      setColor(0, 0, 1); delay(100); setColor(0, 0, 0);  
    }  
  }  
  
  if (!buttonPressed && buttonWasPressed) {  
    buttonWasPressed = false;  
    if (!alertaEnRevision) setColor(0, 1, 0);  
  }  
  
  if (pressCount > 0 && (millis() - pressWindowStart) > PRESS_WINDOW) {  
    int idx = (pressCount - 1) % NUM_TYPES;  
    Serial.printf("[Button] %d pulsación(es) → %s\n", pressCount, EMERGENCY_TYPES[idx]);  
    sendAlert(EMERGENCY_TYPES[idx], EMERGENCY_LCD[idx]);  
    pressCount = 0;  
  }  
  
  delay(20);  
}
```

10. Diagrama de Estados del LED

Estado del Sistema	Color LED
Conectando WiFi / MQTT	AMARILLO
Sistema listo / OK	VERDE
Boton pulsado	AZUL
Enviando alerta	ROJO
Guardia revisando	MAGENTA
Alerta confirmada	VERDE
Falsa alarma	VERDE
Error de envio	ROJO